

DATABASE

laboratory work 2

Normalization

Task 1:

Booking_receipt(booking_id, passenger_full_name, passenger_passport_number, flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name, **seat_numbers**, ticket_price)

1 Normal Form:

The table violates 1NF because the seat_numbers column can contain multiple values in one cell. It contains non-atomic values like "12A, 12B". In 1NF, every attribute must contain a single atomic value. Therefore, seat_numbers is the problem

2 Normal Form:

2NF requires the table to be in 1NF and that all non-key attributes must be fully functionally dependent on the entire primary key (no partial dependencies).

To convert the table to 1NF, a composite primary key like **(booking_id, seat_number)** must be used. However, attributes like **flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, and airline_name** depend only on **booking_id**, while **passenger_full_name** depends only on **passenger_passport number**.

3 Normal Form:

3NF requires the table to be in 2NF and that no non-key attribute depends on another non-key attribute. No transitive dependencies.

departure_city depends on departure_airport_name (booking_id → departure_airport_name → departure_city).

arrival_city depends on arrival_airport_name (booking_id → arrival_airport_name → arrival_city).

airline_name depends on flight_number (booking_id → flight_number → airline_name).

Task 2:

1 Normal Form:

To bring Booking_receipt to 1NF, we eliminate non atomic values from seat_numbers by expanding each seat into its own row. And we create composite primary key (booking_id, seat_number) to uniquely identify rows.

So instead of:

id	name	passport	airline	seat
1	Ali	AB123	FlyArystan	12A, 12B

We do:

id	Name	passport	fligh	D_airport	city	Arr_airport	city	airline	seat	price
1	Ali	AB123	KZ1	AlmatyAir	Almat	AstanaAir	Astan	FlyA	12A	100\$
1	Ali	AB123	KZ1	AlmatyAir	Almat	AstanaAir	Astan	FlyA	12B	100\$

As we can see, id is no more uniquely represents row. So we make it composite by adding seat_number. It creates new problem of breaking 2NF.

2 Normal Form:

To eliminate partial dependencies on the composite key (booking_id, seat_number), attributes dependent only on booking_id are split into a separate booking table, and attributes dependent only on passenger_passport_number are moved to a passenger table.

1. **Bookings_2NF:** (booking_id, flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name)
2. **Passengers_2NF:** (passenger_passport_number, passenger_full_name)
3. **Tickets_2NF:** (booking_id (FK), seat_number, passenger_passport_number (FK), ticket_price)

id	Name	passport	fligh	D_airport	city	Arr_airport	city	airline	seat	price
1	Ali	AB123	KZ1	AlmatyAir	Almat	AstanaAir	Astan	FlyA	12A	100\$
1	Ali	AB123	KZ1	AlmatyAir	Almat	AstanaAir	Astan	FlyA	12B	100\$

As you can see many records are duplicated, 2NF are against this.

So Booking:

Booking id	flight	Dep_airport	Dep_city	Arr_airport	Arr_city	airline
------------	--------	-------------	----------	-------------	----------	---------

1	KZ1	AlmatyAir	Almaty	AstanaAir	Astana	FlyA
---	-----	-----------	--------	-----------	--------	------

Ticket:

Booking_id	Seat_number	passport	Passenger_name	price
1	12A	AB123	Ali	100\$
1	12B	AB123	Ali	100\$

3 Normal Form:

To make it 3NF we remove transitive functional dependencies where non-key attributes depend on other non-key attributes inside.

Airport:

Airport_name	Airport_city
AlmatyAir	Almaty
AstanaAir	Astana

Airline:

Airline id	Airline name
1	FlyArustan

Flight:

Flight number	Airline id	Dep_airport_name	Arr_airport_name
KZ1	1	AlmatyAir	AstanaAir

Passport:

Passport	Full_name
AB123	Ali

Booking:

Booking_id	Flight_number
1	KZ1

Ticket:

Ticket_id	Booking_id	Passport	seat	price
1	1	AB123	12A	100\$
2	2	AB123	12B	100\$

Task 3:

Functional dependencies 2NF:

- (booking_id, seat_number) -> passenger_passport_number, passenger_full_name, ticket_price
- booking_id -> flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name

3NF:

- passenger_passport_number -> passenger_full_name
- dep_airport_name -> departure_city
- arr_airport_name -> arrival_city
- flight_number -> airline_id, departure_airport_name, arrival_airport_name
- airline_name -> airline_id
- booking_id -> flight_number

Task 4:

Flights: flight_id(int), sch_departure_time(timestamp), sch_arrival_time(timestamp), departing_airport_id(int), arriving_airport_id(int), departing_gate(varchar(50)), arriving_gate(varchar(50)), airline_id(int), act_departure_time(timestamp), act_arrival_time(timestamp), created_at(timestamp), updated_at(timestamp) ;

Functional Dependencies:

- flight_id → flight_number, departure_time, arrival_time, created_at, updated_at, act_departure_time, act_arrival_time, departing_gate

All of them are logically parameters of flight.

We need to check whether any attributes in the table depend on these attributes:

departing_airport_id, arriving_airport_id, airline_id;

Their logical attributes (city, country, names, states etc.) are not on the table, so other attributes don't depend on them. They are just FK.

Task 5:



Task5.sql

Code:

Airline

Airline_id	Airline_name
1	FlyArustan
2	Air Astana
3	SCAT
4	Emirates
5	British Airways

Flight

Flight_number	Airline_id	Dep_airport_name	Arr_airport_name
KZ1	1	AlmatyAir	AstanaAir
KU2	2	AlmatyAir	LondonAir
KD3	3	AstanaAir	DubaiAir
UU4	4	LondonAir	USAir
KZ5	1	AstanaAir	AlmatyAir

Airport

Airport_name	Airport_city
AlmatyAir	Almaty
AstanaAir	Astana
LondonAir	London
USAir	New York
DubaiAir	Dubai

Passport

Passport	Full_name
AB123	Ali
CD456	Aruzhan
EF789	Daniyar
GH123	John
IJ456	Elena

Booking

Booking_id	Flight_number
1	KZ1
2	KZ1
3	KD3
4	UU4
5	KZ5

Ticket

Ticket_id	Booking_id	Passport	seat	price
1	1	AB123	12A	100\$
2	2	AB123	12B	100\$
3	3	EF789	03F	280\$
4	4	GH123	21C	190\$
5	5	IJ456	08D	310\$

International Airport Database

This laboratory project designs and implements a relational database architecture for an International Airport operational system. The relational schema is normalized into 10 distinct entities to ensure domain data integrity and eliminate functional redundancies.

TABLES:

Name	Primary Key	Foreignn Key
Airport	airport	---
Airline	airline id	---
Flights	flight id	airline id, departure airport id, arrival airport id
Passengers	passenger id	---
Booking	booking id	passenger id, flight id
Booking flight	booking flight id	Booking id, flight id
Boarding	boarding id	booking id
Baggage	baggage id	booking id
Baggage check	baggage check id	booking id, passenger id
Security check	security check id	passenger id

Connection:

Connections connect tables with foreign and primary key attributes. To understand connections better we can divide them into 3 sections:

Flight connection:

airlines -> flights (1:N) or (1:0): One airline does many flights or might not have any flights for the moment.

airports -> flights (1:N): One airport has many departures and arrivals

Booking:

passengers -> booking (1:N): One passenger can book many tickets.

Booking -> flights (M:N): A single booking can include multiple connecting flights (segments), and many bookings are sold for a single flight.

booking -> booking_changes (1:N) or (0:N): One booking may have many changes, or may not have it.

booking -> boarding (1:1): One booking has exactly one boarding with one seat and time.

Baggage:

booking -> baggage (1:N) or (0:N): In one booking may have many baggages, or may not have it.

booking + passengers -> baggage_checks (1:N) or (0:N): Baggage checks connects passenger and its booking, and there might be more than one baggage checks, or zero.

passengers -> security_check (1:N): Passenger could went from many security checks.

Visualisation:



